Universidad Mariano Gálvez de Guatemala
Facultad de Ingeniería en Sistemas de Información y Ciencias de la Computación
Ingeniería en Sistemas de Información

Entregable 4: Scripts de simulación de eventos
Sistema de monitoreo perimetral urbano para la Municipalidad de San Juan Sacatepéquez

[Nombre del integrante 1] [Carné]
[Nombre del integrante 2] [Carné]
[Nombre del integrante 3] [Carné]

Arquitectura de Computadores II, sección 040
[Nombre del catedrático]
31 de octubre de 2026

Índice

Scripts de simulación de eventos 1

Propósito del documento 1

Herramientas de simulación 1

Organización de los scripts 2

Sectores simulados 2

Generación de eventos por sensor 3

Tasas y comportamiento en el tiempo 4

Eventos inválidos 6

Envío y reintentos 6

Modos de ejecución 8

Escenarios guionados 8

Reporte ciudadano manual 9

Placas buscadas y evaluación 10

Plan de carga con JMeter 10

Resultados de la simulación 11

Referencias 12

Scripts de simulación de eventos

Propósito del documento

El proyecto no usa sensores físicos: los sensores urbanos se emulan con programas que reproducen su comportamiento, frecuencia, ubicación y tipo de evento. Este documento describe los scripts de simulación, cómo generan los datos de cada sensor, cómo se ejecutan y qué resultados producen. Corresponde al cuarto entregable de la sección 8 del enunciado. Los scripts están en la carpeta simulador del repositorio y el plan de carga de JMeter en pruebas/jmeter.

Herramientas de simulación

Tabla 1

Herramientas sugeridas por el enunciado y uso en el proyecto

Herramienta sugerida	Uso en el proyecto
Scripts Python	Herramienta principal: generan los cinco tipos de eventos, el tráfico normal, los picos de carga, los escenarios guionados y los reportes manuales
Artillery / JMeter	JMeter 5.5 para pruebas de carga HTTP de la capa de ingestión (plan pruebas/jmeter/carga.jmx)
Apache Kafka Producer CLI	No se usa para generar eventos: los eventos entran por la API REST para pasar por la validación, igual que en un despliegue real. Las herramientas de línea de comandos de Kafka se usaron para inspeccionar los topics durante las pruebas
CSV predefinidos + Randomizer	Se reemplazan por config/sectores.json (sectores con coordenadas reales, pesos y radios) y por generadores aleatorios con distribución por sector y por hora

Organización de los scripts

Tabla 2

Archivos del simulador

Archivo	Función
generadores.py	Crea los eventos de cada tipo de sensor con el contrato común, calcula el HMAC de las placas y daña eventos a propósito
envio.py	Envía lotes a NGINX con reintentos y lleva los contadores del resumen
simulador.py	Programa principal: modos normal, pico y escenario
escenarios.py	Los nueve escenarios guionados y su alerta esperada
reportar.py	Reporte ciudadano manual e interactivo
cargar_buscados.py	Carga en Redis los hashes de las placas reportadas como robadas
evaluar.py	Compara alertas esperadas y obtenidas por escenario

Todos los scripts se ejecutan dentro de un contenedor (no requieren Python instalado en la máquina) con el comando docker compose run --rm simulador seguido del script y sus opciones.

Sectores simulados

Los eventos se ubican en los siete sectores de config/sectores.json (Tabla 3). El peso es la probabilidad de que un evento ocurra en ese sector y el radio define el círculo alrededor del centro donde se dispersan los eventos, para que el mapa de calor se vea natural. Se validan los límites del municipio: latitud entre 14.60 y 14.90, longitud entre -90.75 y -90.50.

Tabla 3

Sectores del municipio

Sector	Nombre	Tipo	Latitud	Longitud	Peso	Radio	Escolar	Límite
SJS-01	Casco urbano	comercial	14.71840	-90.64340	0.20	600 m	Sí	40 km/h
SJS-02	Sajcavillá	mixto	14.70645	-90.62770	0.14	700 m	No	50 km/h
SJS-03	Cruz Blanca	residencial	14.73330	-90.66670	0.20	800 m	Sí	50 km/h
SJS-04	Lo de Mejía	corredor	14.69256	-90.60027	0.12	900 m	No	60 km/h
SJS-05	Ciudad Quetzal	urbano denso	14.70313	-90.58035	0.20	800 m	Sí	50 km/h
SJS-06	Villas del Quetzal	residencial	14.70831	-90.55144	0.09	600 m	No	40 km/h
SJS-07	Montúfar	rural	14.79956	-90.63965	0.05	1500 m	No	60 km/h

Generación de eventos por sensor

Cada evento lleva un identificador único (UUID), la hora en que ocurrió, la hora de envío (que se fija justo antes de enviarlo, para medir la latencia real), los datos del dispositivo, la ubicación y los datos propios del sensor (Tabla 4). La Figura 1 muestra un evento completo.

Tabla 4

Datos generados por tipo de sensor

Sensor	Datos y valores	Comportamiento en tráfico normal
Botón de pánico	alertType: PANICO, EMERGENCIA_MEDICA o INCENDIO; origin: BOTON_FIJO o APP_MOVIL; userRef: seudónimo solo si viene de la app	80 % pánico, 15 % emergencia médica, 5 % incendio
Cámara LPR	plateHash (HMAC-SHA256), platePartial (P-***BCD), estimatedSpeedKmh, direction (ENTRADA o SALIDA), vehículo (tipo, color, modelo)	Placa sintética guatemalteca nueva en cada lectura; 0.2 % sobre el límite
Radar de velocidad	speedKmh, limitKmh, headingDegrees, anomalyType: NINGUNA, EXCESO_VELOCIDAD o TRAYECTORIA_ANOMALA	0.2 % con exceso y 0.1 % con trayectoria anómala
Sensor acústico	soundType: DISPARO, EXPLOSION o VIDRIO_ROTO; decibels; criticalProbability	Probabilidad casi siempre por debajo de 0.7
Reporte ciudadano	reportType: ACCIDENTE, INCENDIO o ALTERCADO; message (texto libre, máximo 280 caracteres); origin: USUARIO, APP o PUNTO_FISICO; accuracyMeters	Mensajes sintéticos de ejemplo por tipo

Nota. Las anomalías son raras a propósito en el tráfico normal: con anomalías frecuentes aparecerían alertas graves por puro azar. Las alertas importantes vienen de los escenarios guionados.

Figura 1

Ejemplo de evento de botón de pánico

```
{
  "schemaVersion": "1.0",
  "eventId": "8f14e45f-ceea-4e1b-9c2a-1a2b3c4d5e6f",
  "eventType": "security.panic_button.pressed",
  "timestamp": "2026-10-28T22:05:13.412-06:00",
  "emittedAt": "2026-10-28T22:05:13.430-06:00",
  "scenarioId": null,
  "device": { "deviceId": "PAN-SJS05-012", "deviceType": "panic_button",
              "model": "SafeCity-PB100", "firmwareVersion": "1.2.0" },
  "location": { "zone": "SJS-05", "zoneName": "Ciudad Quetzal",
                "city": "San Juan Sacatepéquez",
                "latitude": 14.70391, "longitude": -90.58102 },
  "data": { "alertType": "PANICO", "origin": "APP_MOVIL", "userRef": "u_7c1e9a" },
  "metadata": { "batteryLevel": 84, "signalStrength": -67, "protocol": "MQTT" }
}
```

Nota. Contrato de eventos del sistema, basado en el ejemplo del catedrático (device, location, data y metadata).

Tasas y comportamiento en el tiempo

Cada segundo, el simulador calcula cuántos eventos de cada tipo genera: toma la tasa por segundo, la multiplica por el multiplicador del modo y por un factor según la

hora del día, y la convierte en un número entero con una moneda al aire para la parte fraccionaria (Figura 2). Hay más tráfico vehicular en las horas pico (06:00 a 09:00 y 17:00 a 20:00) y más pánicos y disparos de noche (19:00 a 02:00), con un factor de 1.5.

Figura 2

Tasas por segundo y factor horario

```
# Tasas por segundo en todo el municipio (sección 9 del plan)
TASAS = {
    "lpr": 3,
    "velocidad": 2,
    "acustico": 2 / 60,
    "panico": 1 / 60,
    "reporte": 2 / 60,
}
GENERADORES = {
    "lpr": evento_lpr, "velocidad": evento_velocidad, "acustico": evento_acustico,
    "panico": evento_panico, "reporte": evento_reporte,
}



def factor_hora(tipo, hora):
    """Más tráfico en horas pico; más pánico y disparos de noche."""
    if tipo in ("lpr", "velocidad") and (6 <= hora < 9 or 17 <= hora < 20):
        return 1.5
    if tipo in ("panico", "acustico") and (hora >= 19 or hora < 2):
        return 1.5
    return 1.0



def cuantos(tasa):
    """Cantidad de eventos en este segundo: la parte entera más una moneda al aire por la
fracción."""
    entera = int(tasa)
    return entera + (1 if random.random() < tasa - entera else 0)
```

Nota. Fragmento del archivo simulador/simulador.py, líneas 26 a 53.

Tabla 5

Tasas de simulación

Sensor	Normal (todo el municipio)	Pico (multiplicador 20)
Cámara LPR	3 por segundo	60 por segundo
Radar de velocidad	2 por segundo	40 por segundo
Sensor acústico	2 por minuto	40 por minuto
Botón de pánico	1 por minuto	20 por minuto
Reporte ciudadano	2 por minuto	40 por minuto

Eventos inválidos

Para probar la validación y la cola de mensajes muertos (dead-letter), el simulador daña a propósito el 1.5 % de los eventos con alguno de los defectos de la Tabla 6. El ingestor los rechaza con un motivo legible y los publica en el topic events.invalid.

Tabla 6

Defectos introducidos a propósito

Defecto	Motivo de rechazo esperado
Placa en claro dentro de data (privacidad)	PLACA_EN_CLARO
Sector inexistente (SJS-99)	ZONA_DESCONOCIDA
Latitud fuera del municipio	COORDENADAS_FUERA_DE_LIMITES
Fecha en el futuro (año 2099)	TIMESTAMP_EN_EL_FUTURO
Falta el bloque device	CAMPO_INVALIDO:device
Mensaje de reporte de 300 caracteres	MENSAJE_DEMASIADO_LARGO
Hash de placa con formato incorrecto	PLATE_HASH_INVALIDO
Versión de esquema desconocida	CAMPO_INVALIDO:schemaVersion

Envío y reintentos

Los eventos se envían en lotes de hasta 50 a POST /events/batch a través de NGINX, al menos una vez por segundo, con varios hilos en paralelo. Si un envío falla (por ejemplo, HTTP 503 porque Kafka no responde), se reintenta con espera creciente de 1, 2, 4 y 8 segundos, hasta cinco intentos; si aun así falla, el lote se cuenta como perdido (Figura 3). Al final de cada ejecución se imprime un resumen con los eventos enviados, aceptados, rechazados, perdidos, los reintentos y los eventos por segundo.

Figura 3

Envío con reintentos y espera creciente

```
def enviar(eventos):
    """Envía una lista de eventos a /events/batch. Devuelve la respuesta JSON o None si se
perdió."""
    if not eventos:
        return None
    # emittedAt = momento real de envío; el correlador lo usa para medir la latencia
    momento = ahora_iso()
    for evento in eventos:
        if isinstance(evento, dict) and evento.get("emittedAt") is None:
            evento["emittedAt"] = momento

    espera = 1
    for intento in range(1, INTENTOS + 1):
        try:
            respuesta = sesion().post(f"{URL}/events/batch", json=eventos, timeout=15)
            if respuesta.status_code == 200:
                cuerpo = respuesta.json()
                sumar("enviados", len(eventos))
                sumar("aceptados", cuerpo["aceptados"])
                sumar("rechazados", cuerpo["rechazados"])
                return cuerpo
            motivo = f"HTTP {respuesta.status_code}"
        except requests.RequestException as error:
            motivo = type(error).__name__
        if intento < INTENTOS:
            log.warning("Envío fallido (%s), intento %d de %d, reintento en %d s", motivo,
intento, INTENTOS, espera)
            sumar("reintentos", 1)
            time.sleep(espera)
            espera *= 2

    log.error("Lote de %d eventos PERDIDO después de %d intentos", len(eventos), INTENTOS)
    sumar("enviados", len(eventos))
    sumar("perdidos", len(eventos))
    return None
```

Nota. Fragmento del archivo simulador/envio.py, líneas 37 a 69.

Modos de ejecución

Figura 4

Comandos de los scripts de simulación

```
# Tráfico normal (hasta Ctrl+C o durante N segundos)
docker compose run --rm simulador simulador.py normal --duracion 120

# Prueba de carga: multiplica las tasas
docker compose run --rm simulador simulador.py pico --multiplicador 20 --duracion 60 --hilos 16

# Escenarios guionados (todos a la vez o uno solo)
docker compose run --rm simulador simulador.py escenario todos --velocidad 2
docker compose run --rm simulador simulador.py escenario ESC-ROBO-01

# Utilidades
docker compose run --rm simulador cargar_buscados.py
docker compose run --rm simulador reportar.py
docker compose run --rm simulador evaluar.py --minutos 5
```

Nota. Los scripts se ejecutan en un contenedor; no se requiere Python en la máquina.

Escenarios guionados

Los escenarios reproducen incidentes concretos para demostrar y evaluar la correlación (Tabla 7). Los eventos se ubican en el centro exacto del sector y llevan un identificador de escenario (scenarioId), que el sistema solo usa para la evaluación. Con la opción --velocidad se dividen las esperas entre eventos (velocidad 2 = el doble de rápido). ESC-ROBO-01 y ESC-FALSO-01 son los dos ejemplos de la sección 2.4 del enunciado.

Tabla 7

Escenarios guionados

Escenario	Sector	Secuencia	Alerta esperada
ESC-ROBO-01	Ciudad Quetzal	Pánico; 40 s después LPR de salida a 85 km/h; 20 s después radar a 92 km/h	ROBO_EN_CURSO
ESC-ARMADO-01	Casco urbano	Disparo con probabilidad 0.9; 30 s después pánico	ATAQUE_ARMADO
ESC-ACCIDENTE-01	Lo de Mejía	Trayectoria anómala; 60 s después reporte de accidente	ACCIDENTE_VIAL
ESC-INCENDIO-01	Villas del Quetzal	Pánico de incendio; 45 s después reporte de incendio	INCENDIO_CONFIRMADO
ESC-TRAMO-01	Lo de Mejía a Sajcavillá	Misma placa en dos cámaras con 120 s de diferencia (unos 100 km/h)	VELOCIDAD_ENTRE_CAMARAS
ESC-ESCOLAR-01	Casco urbano	Radar a 45 km/h en horario escolar	ZONA_ESCOLAR
ESC-BUSCADO-01	Cruz Blanca	LPR de una placa de la lista de robados	VEHICULO_REPORTADO
ESC-FALSO-01	Montúfar	Un solo pánico, nada más	PANICO_NO_CONFIRMADO
ESC-RUIDO-01	Sajcavillá	Pánico y vidrio roto con probabilidad 0.4	PANICO_NO_CONFIRMADO

Figura 5

Definición de dos escenarios

```
        "ESC-ROBO-01": [
            (0, lambda: evento_panico("SJS-05", "PANICO", e, centro=True)),
            (40, lambda: evento_lpr("SJS-05", velocidad=85, direccion="SALIDA", escenario=e,
centro=True)),
            (20, lambda: evento_velocidad("SJS-05", 92, "EXCESO_VELOCIDAD", e, centro=True)),
        ],
        "ESC-ARMADO-01": [
            (0, lambda: evento_acustico("SJS-01", "DISPARO", 0.9, e, centro=True)),
            (30, lambda: evento_panico("SJS-01", "PANICO", e, centro=True)),
        ],
```

Nota. Fragmento del archivo simulador/escenarios.py, líneas 42 a 50.

Reporte ciudadano manual

El script reportar.py cumple la entrada manual de la sección 2.1 del enunciado: pregunta en la terminal el sector, el tipo de evento (accidente, incendio o altercado) y

un mensaje libre de hasta 280 caracteres, arma el evento con el mismo contrato y lo envía como lo haría la app del vecino. Al terminar muestra el identificador del reporte y el ingestor que lo atendió. Dos reportes de accidente en el mismo sector dentro de la ventana generan la alerta ACCIDENTE_VIAL.

Placas buscadas y evaluación

cargar_buscados.py calcula el HMAC de una lista de diez placas sintéticas reportadas como robadas y las guarda en el conjunto placas_buscadas de Redis; es el único script del simulador que habla directamente con Redis. evaluar.py compara, para cada escenario, la alerta esperada con las alertas que el correlador guardó en PostgreSQL, muestra una tabla con aciertos, faltantes y alertas adicionales, y guarda el resultado en la tabla evaluation_results.

Plan de carga con JMeter

El plan pruebas/jmeter/carga.jmx envía lotes de eventos válidos (cámaras LPR y radares) a POST /events/batch. Un preprocesador en Groovy arma cada lote con identificadores y horas reales. Los parámetros se pasan con la opción -J (Tabla 8).

Tabla 8

Parámetros del plan de JMeter

Parámetro	Valor por defecto	Significado
hilos	10	Usuarios virtuales enviando a la vez
rampa	5	Segundos para arrancar todos los hilos
duracion	60	Segundos de prueba
tamano_lote	50	Eventos por petición (máximo 500)
host / puerto	localhost / 8080	Destino (dentro de la red de Docker: nginx / 80)

Figura 6

Ejecución del plan de JMeter con Docker

```
docker run --rm --network monitoreo-sjs_default -v "${PWD}\pruebas\jmeter:/pruebas"
justb4/jmeter:5.5 `
  -n -t /pruebas/carga.jmx -Jhost=nginx -Jpuerto=80 -Jhilos=16 -Jduracion=60 -l
/pruebas/resultados.jtl
```

Nota. Comando para PowerShell documentado en el README.

Resultados de la simulación

Tabla 9

Resultados obtenidos con los scripts

Ejecución	Resultado
Tráfico normal, 2 minutos	610 eventos enviados, 597 aceptados, 13 rechazados a propósito, 0 perdidos, 5.1 eventos por segundo
Pico ×100, 2 minutos	60 998 enviados, 60 077 aceptados, 0 perdidos, 508 eventos por segundo; latencia p95 de alertas de 976 ms
Escenarios guionados	9 de 9 alertas esperadas
JMeter, 16 hilos, 30 s	541 lotes por segundo (unos 27 000 eventos por segundo), 0 errores
Kafka caído 32 s durante un pico ×20	Los reintentos evitaron cualquier pérdida (0 perdidos)

Referencias

Krawczyk, H., Bellare, M., & Canetti, R. (1997). *HMAC: Keyed-hashing for message authentication* (RFC 2104). Internet Engineering Task Force. https://doi.org/10.17487/RFC2104